Báo cáo bài tập lớn: Kỹ thuật lập trình

Trần Đức Anh — B24DCVT021

BÁO CÁO BÀI TẠP LỚN: KỸ THUẬT LẠP TRÌNH

ĐỀ TÀI: QUẢN LÝ THIẾT BỊ DI ĐỘNG

Phần thực hiện: Thiết bị đầu cuối IMEI và các thư viện dùng chung/liên quan

Sinh viên thực hiện: Trần Đức Anh
Mã sinh viên: B24DCVT021
Lớp: D24CQVT01-B
Học viện: Học viện Công nghệ Bưu chính Viễn thông (PTIT)
Học phần: Kỹ thuật lập trình (KTLT)
Năm học: 2026–2027
Nhánh phát triển: DucAnh-B24DCVT021

MỤC LỤC

- Mở đầu
- Chương 1. Giới thiệu và phân tích yêu cầu
- Chương 2. Môi trường và công cụ lập trình
- Chương 3. Thiết kế giải pháp
- Chương 4. Kiến thức mới tự tìm hiểu và vận dụng
- Kết luận
- Tài liệu tham khảo

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tiếng Anh	Ý nghĩa
BTS	Base Transceiver Station	Trạm thu phát sóng di động
CRUD	Create, Read, Update, Delete	Thêm, xem/tìm, sửa, xóa dữ liệu
EIR	Equipment Identity Register	Cơ sở đăng ký nhận dạng thiết bị
IDE	Integrated Development Environment	Môi trường phát triển tích hợp
IMEI	International Mobile Equipment Identity	Mã nhận dạng thiết bị di động quốc tế

Từ viết tắt	Tiếng Anh	Ý nghĩa
KTLT	Programming Techniques	Kỹ thuật lập trình
Luhn	Luhn Mod-10 algorithm	Thuật toán kiểm tra tổng chữ số
OOP	Object-Oriented Programming	Lập trình hướng đối tượng
SIM	Subscriber Identity Module	Mô-đun nhận dạng thuê bao
TAC	Type Allocation Code	Mã phân bổ loại thiết bị

MỞ ĐẦU

Trong các hệ thống viễn thông, việc quản lý thuê bao không chỉ bao gồm thông tin khách hàng và hợp đồng, mà còn bao gồm thiết bị đầu cuối đang sử dụng số thuê bao. Mỗi thiết bị di động có một mã nhận dạng IMEI. Mã này hỗ trợ nhận biết thiết bị, tra cứu trạng thái, xử lý trường hợp thiết bị bị mất hoặc bị khóa mạng, và liên kết thiết bị với các nghiệp vụ hỗ trợ khách hàng.

Bài tập lớn xây dựng một hệ thống console bằng C++ để mô phỏng một phần hệ thống quản lý thuê bao di động. Phần thực hiện của sinh viên tập trung vào hai use case: quản lý hợp đồng dịch vụ (HopDong) và quản lý thiết bị đầu cuối IMEI (ThietBiIMEI). Bên cạnh đó, phần này xây dựng các thư viện dùng chung cho ngày tháng, chuẩn hóa dữ liệu, nhập liệu, hiển thị, ngoại lệ, xử lý file, danh bạ BTS và lưu trữ dữ liệu.

Báo cáo được trình bày theo cấu trúc trong đề bài Kỹ thuật lập trình:

1. Chương 1 giới thiệu bài toán và phân tích yêu cầu.
2. Chương 2 trình bày môi trường, công cụ và thư viện chuẩn đã sử dụng.
3. Chương 3 trình bày thiết kế giải pháp, lưu trữ, rủi ro, kiểm thử và hướng mở rộng.
4. Chương 4 trình bày kiến thức mới tự tìm hiểu, nguồn tham khảo, cách áp dụng và lợi ích.

Ứng dụng hiện tại là sản phẩm học tập chạy cục bộ. Ứng dụng không kết nối trực tiếp đến mạng di động thật, EIR thật, HLR/HSS, lõi 5G hay cơ sở dữ liệu của nhà mạng.

Chương 1. Giới thiệu và phân tích yêu cầu

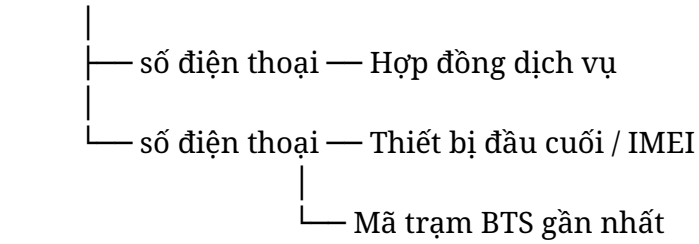
1.1. Mô tả bài toán

Nhà mạng cần quản lý nhiều nhóm dữ liệu liên quan với nhau: khách hàng, SIM, số thuê bao, hợp đồng, gói cước, thiết bị đầu cuối, trạm BTS, nạp tiền và hóa đơn. Nhân viên cần có khả năng thêm mới, tra cứu, cập nhật, khóa hoặc xóa dữ liệu theo đúng quy tắc nghiệp vụ.

Trong phạm vi nhánh phát triển DucAnh-B24DCVT021, hệ thống mô phỏng hai nhóm nghiệp vụ chính:

- **Quản lý hợp đồng dịch vụ (HopDong):** lưu mã hợp đồng, mã khách hàng, số điện thoại, mã gói cước, ngày đăng ký, ngày hết hạn, loại hợp đồng, trạng thái và giá trị gói.
- **Quản lý thiết bị đầu cuối (ThietBiIMEI):** lưu IMEI, tên thiết bị, hãng sản xuất, số điện thoại gắn với thiết bị, ngày kích hoạt, trạng thái thiết bị và trạm BTS gần nhất.

Mối liên hệ cơ bản được mô hình hóa như sau:



Mục tiêu của phần thực hiện là tạo ra các lớp nghiệp vụ độc lập, có thể kiểm tra tự động, có thể lưu và nạp dữ liệu, đồng thời giữ cho giao diện console không chứa toàn bộ quy tắc nghiệp vụ.

1.2. Các chức năng thực hiện trong chương trình

1.2.1. Use case UC01: Quản lý hợp đồng dịch vụ

Các chức năng chính của HopDongMenu gồm:

1. **Thêm hợp đồng:** nhập và kiểm tra mã hợp đồng, mã khách hàng, số điện thoại, mã gói cước, ngày đăng ký, ngày hết hạn, loại hợp đồng, trạng thái và giá trị gói.
2. **Xem danh sách:** hiển thị các hợp đồng dưới dạng bảng console.
3. **Tìm kiếm:** tìm theo mã hợp đồng, số điện thoại hoặc mã khách hàng.
4. **Sắp xếp:** sắp xếp theo ngày đăng ký hoặc giá trị gói.
5. **Cập nhật:** thay đổi thông tin cần thiết, gia hạn hoặc thay đổi trạng thái.
6. **Tạm dừng, kích hoạt lại và chấm dứt:** xử lý vòng đời hợp đồng.
7. **Xóa:** yêu cầu xác nhận trước khi xóa và đồng bộ dữ liệu xuống file.

1.2.2. Use case UC02: Quản lý thiết bị đầu cuối IMEI

Các chức năng chính của ThietBiIMEIMenu gồm:

1. **Thêm thiết bị:** nhập IMEI và thông tin thiết bị; IMEI phải hợp lệ theo Luhn Mod-10.
2. **Xem danh sách:** hiển thị thiết bị dưới dạng bảng.
3. **Tìm kiếm:** tìm theo IMEI, số điện thoại hoặc mã BTS.
4. **Xem danh sách khóa mạng:** lọc các thiết bị có trạng thái KhoaMang.
5. **Gán SIM:** gắn một số điện thoại hợp lệ vào thiết bị.
6. **Gỡ SIM:** đưa thiết bị về trạng thái chưa gán số điện thoại (ChuaGan).

7. **Cập nhật BTS:** thay đổi mã trạm BTS gần nhất.
8. **Khóa hoặc mở khóa mạng:** chuyển trạng thái giữa KhoaMang và HoatDong.
9. **Xóa thiết bị:** xóa bản ghi sau khi xác nhận.

1.3. Phân tích yêu cầu dữ liệu và kiểm tra hợp lệ

1.3.1. Yêu cầu đối với IMEI

IMEI trong phạm vi bài tập phải:

- có đúng 15 ký tự;
- chỉ chứa các chữ số từ 0 đến 9;
- không phải chuỗi toàn số 0;
- thỏa mãn thuật toán Luhn Mod-10;
- không trùng với IMEI đã có trong DataStore<ThietBiIMEI>.

Trong cấu trúc IMEI hiện đại, 15 chữ số thường được mô tả gồm 8 chữ số TAC, 6 chữ số SNR và 1 chữ số kiểm tra. Chương trình lưu toàn bộ 15 chữ số như một mã định danh duy nhất, không tách TAC và SNR thành các trường riêng.

1.3.2. Yêu cầu đối với số điện thoại

Số điện thoại di động phải có 10 chữ số và bắt đầu bằng một trong các đầu số 03, 05, 07, 08, 09. Với thiết bị chưa gán SIM, hệ thống cho phép giá trị đặc biệt ChuaGan.

1.3.3. Yêu cầu đối với ngày tháng

Thư viện Date kiểm tra ngày hợp lệ theo lịch Gregory, bao gồm năm nhuận và số ngày trong từng tháng. Ngày hết hạn hợp đồng không được nhỏ hơn ngày đăng ký. Ngày sinh thuê bao được kiểm tra theo giới hạn tuổi được cấu hình trong thư viện.

1.3.4. Yêu cầu đối với trạng thái thiết bị

Thiết bị chỉ được nhận các trạng thái đã định nghĩa:

- HoatDong — thiết bị đang hoạt động.
- KhoaMang — thiết bị bị khóa mạng hoặc nằm trong danh sách đen mô phỏng.
- TamKhoa — thiết bị tạm khóa.
- ChuaGan — giá trị này dùng cho trạng thái chưa gán số điện thoại, không phải một trạng thái IMEI độc lập.

1.4. Yêu cầu chung của hệ thống

Ngoài các yêu cầu nghiệp vụ, hệ thống cần đáp ứng:

- **Tính chính xác:** dữ liệu sai định dạng, mã trùng và giá trị không hợp lệ phải bị từ chối.
- **Tính nhất quán:** khi thêm, sửa hoặc xóa dữ liệu, bộ nhớ và file phải được đồng bộ.

- **Tính dễ sử dụng:** menu console phải có lựa chọn rõ ràng và thông báo lỗi dễ hiểu.
- **Tính kiểm thử:** các quy tắc quan trọng phải có thể kiểm tra mà không cần nhập tay.
- **Tính bảo trì:** mã nguồn được chia thành header và implementation theo trách nhiệm.
- **Tính an toàn file:** quá trình ghi dữ liệu phải giảm nguy cơ làm hỏng file chính khi bị gián đoạn.

1.5. Kết luận chương

Chương 1 đã giới thiệu bài toán quản lý thiết bị di động và xác định phạm vi thực hiện gồm quản lý hợp đồng, thiết bị IMEI và các thư viện dùng chung. Các yêu cầu chính tập trung vào CRUD, xác thực IMEI bằng Luhn, kiểm tra số điện thoại, quản lý trạng thái thiết bị, liên kết SIM và BTS, cùng với lưu trữ dữ liệu an toàn.

Chương 2. Môi trường và công cụ lập trình

2.1. Môi trường phát triển

Thành phần	Công cụ hoặc công nghệ	Mục đích sử dụng
Hệ điều hành	Debian GNU/Linux 13	Biên dịch, chạy và kiểm thử chương trình console
Ngôn ngữ	C++	Xây dựng mô hình nghiệp vụ và giao diện console
Trình biên dịch	GCC/G++	Biên dịch mã nguồn thành chương trình thực thi
Chuẩn ngôn ngữ	C++11 theo Makefile	Bảo đảm mã nguồn dùng được với chuẩn C++ được cấu hình
IDE / trình soạn thảo	Visual Studio Code, Neovim	Viết mã, tổ chức file và tích hợp terminal
Công cụ xây dựng	GNU Make	Tự động hóa lệnh biên dịch và chạy test
Quản lý phiên bản	Git và GitHub	Tạo nhánh, xem lịch sử, review và tích hợp mã nguồn
Kiểm thử	tests/test_runner.cpp	Chạy các kiểm tra hồi quy bằng assertion
Gỡ lỗi	Terminal, warning của GCC và debugger của IDE	Theo dõi lỗi biên dịch và lỗi logic

Makefile hiện tại sử dụng các cờ -std=c++11 -Wall -Wextra. Dự án có thể được mở bằng Visual Studio Code hoặc Neovim, nhưng việc biên dịch chính thức được thực hiện qua GCC/G++ và Make.

2.2. Công cụ lập trình và phối hợp nhóm

2.2.1. Visual Studio Code và Neovim

Visual Studio Code hỗ trợ quản lý nhiều file, terminal tích hợp, cấu hình build và debug. Neovim được sử dụng như một trình soạn thảo nhẹ cho các thao tác chỉnh sửa trực tiếp trong terminal. Hai công cụ chỉ đóng vai trò môi trường làm việc; quy tắc đúng sai của chương trình được xác định bởi compiler và test suite.

2.2.2. GCC/G++ và GNU Make

GCC kiểm tra cú pháp, kiểu dữ liệu, liên kết các translation unit và tạo executable. GNU Make lưu các quan hệ phụ thuộc giữa target và source, giúp chỉ biên dịch lại khi cần.

Các lệnh chính của dự án:

```
# Biên dịch chương trình chính
make all

# Chạy bộ kiểm thử
make test

# Chạy chương trình console
make run

# Xóa executable và object file
make clean
```

2.2.3. Git và GitHub

Mỗi thành viên phát triển trên một nhánh riêng theo tên và mã sinh viên. Quy trình làm việc gồm:

1. Đọc yêu cầu và xác định module phụ trách.
2. Phát triển trên nhánh riêng.
3. Chạy test và kiểm tra git diff --check.
4. Review thay đổi và kiểm tra ảnh hưởng đến module liên quan.
5. Tích hợp vào nhánh chung khi thay đổi đã được xác minh.

Nhánh hiện tại là DucAnh-B24DCVT021.

2.3. Công cụ gỡ lỗi và kiểm thử

Việc gỡ lỗi được thực hiện qua nhiều tầng:

- GCC với -Wall -Wextra để phát hiện cảnh báo.
- Terminal để tái hiện lỗi bằng input cụ thể.
- Debugger của IDE hoặc GDB khi cần theo dõi từng bước.
- Bộ test độc lập trong tests/test_runner.cpp để kiểm tra các quy tắc nghiệp vụ.
- git diff --check để phát hiện lỗi whitespace.

Bộ test không sử dụng các mock nông để bỏ qua parser hoặc persistence. Các test gọi trực tiếp các lớp, thuật toán, serialization và DataStore<T> của dự án.

2.4. Lập trình hướng đối tượng áp dụng

Kỹ thuật	Cách áp dụng
Đóng gói	Dữ liệu của HopDong và ThietBiIMEI được quản lý trong lớp; getter/setter kiểm soát truy cập
Kế thừa	HopDong và ThietBiIMEI kế thừa từ Entity
Trừu tượng	Entity quy định các hành vi hiển thị và serialization chung
Đa hình	DataStore<T> làm việc với các kiểu thực thể có cùng hợp đồng phương thức; các phương thức display/serialization được override
Template	DataStore<T> dùng lại logic CRUD và file cho nhiều loại thực thể
Ngoại lệ	Các lỗi IMEI, ngày, số điện thoại, mã trùng, không tìm thấy và file được biểu diễn bằng exception riêng

2.5. Danh sách thư viện chuẩn C++ đã dùng và mục tiêu

Thư viện	Thành phần sử dụng	Mục tiêu
<iostream>	std::cin, std::cout	Nhập và hiển thị dữ liệu console
<string>	std::string	Lưu mã IMEI, tên thiết bị, số điện thoại, trạng thái và đường dẫn
<vector>	std::vector	Lưu danh sách đối tượng trong DataStore<T>
<sstream>	std::ostringstream, string parsing	Ghép chuỗi file và xử lý dữ liệu dạng chuỗi
<iomanip>	std::setw, căn lề	Hiển thị bảng thiết bị và hợp đồng

Thư viện	Thành phần sử dụng	Mục tiêu
<fstream>	std::ifstream, std::ofstream	Đọc và ghi dữ liệu file
<algorithm>	thuật toán chuẩn và sắp xếp	Lọc, sắp xếp và xử lý danh sách
<functional>	std::function	Truyền predicate và comparator cho DataStore<T>
<utility>	std::pair	Biểu diễn cặp nhãn–giá trị trong detail card
<ctime>	thời gian hệ thống	Lấy ngày hiện tại cho Date::now()
<cctype>	kiểm tra ký tự	Hỗ trợ chuẩn hóa và kiểm tra dữ liệu ký tự
<stdexcept>	std::runtime_error	Làm lớp cơ sở cho các lỗi miền nghiệp vụ
<cstdio>	std::rename và thao tác file	Thay file tạm bằng file dữ liệu chính

Dự án không thêm thư viện bên ngoài cho các chức năng mà thư viện chuẩn và mã nguồn ngắn có thể xử lý được.

2.6. Kết luận chương

Chương 2 đã trình bày môi trường Debian 13, GCC/G++, C++11, Visual Studio Code, Neovim, GNU Make, Git/GitHub và bộ test assertion. Các thư viện chuẩn được lựa chọn theo đúng trách nhiệm của từng module, giúp mã nguồn ít phụ thuộc, dễ biên dịch và dễ kiểm tra.

Chương 3. Thiết kế giải pháp

3.1. Giải pháp tổng thể

Hệ thống được tổ chức theo các tầng có trách nhiệm riêng:

Menu console
↓
Domain models: HopDong, ThietBiIMEI
↓
Shared validation/date/display helpers
↓
DataStore<T> và FileIO

↓
Các file dữ liệu trong thư mục data/

Giải pháp này được lựa chọn vì phù hợp với quy mô bài tập, giúp mỗi lớp có một trách nhiệm rõ ràng và cho phép kiểm thử model mà không cần chạy toàn bộ giao diện. InputHelper xử lý tương tác bàn phím; Normalized và các model xử lý tính hợp lệ; DataStore<T> xử lý collection và persistence.

3.2. Tổ chức mã nguồn

```
src/
├── main.cpp
└── lib/
    ├── shared/
    │   ├── Entity.h
    │   ├── Exceptions.h
    │   ├── Date.h / Date.cpp
    │   ├── fileio.h / fileio.cpp
    │   ├── Normalized.h / Normalized.cpp
    │   ├── InputHelper.h / InputHelper.cpp
    │   ├── DisplayHelper.h
    │   ├── DataStore.h
    │   └── BTSRegister.h / BTSRegister.cpp
    ├── models/
    │   ├── hopdong.h / hopdong.cpp
    │   └── imei.h / imei.cpp
    ├── menus/
    │   ├── HopDongMenu.h / HopDongMenu.cpp
    │   └── ThietBiIMEIMenu.h / ThietBiIMEIMenu.cpp
    └── stubs/
        ├── goicuoc.*
        ├── thuebao.*
        ├── naptien.*
        └── hoadon.*
```

Tầng stubs là điểm tích hợp cho các module của thành viên khác. Trong trạng thái hiện tại, các module đó vẫn là điểm nối đang được tích hợp; báo cáo này không coi chúng là nghiệp vụ đã hoàn thiện trong nhánh IMEI.

3.3. Thiết kế lớp ThietBiIMEI

ThietBiIMEI kế thừa Entity và có các trường:

Trường	Ý nghĩa
id	IMEI, kế thừa từ Entity

Trường	Ý nghĩa
tenThietBi	Tên hoặc model thiết bị
hangSanXuat	Hãng sản xuất
soDienThoai	Số thuê bao gắn với thiết bị hoặc ChuaGan
ngayKichHoat	Ngày kích hoạt
trangThai	Trạng thái thiết bị
tramBTSGanNhat	Mã trạm BTS gần nhất

Các phương thức nghiệp vụ chính:

- validateLuhn() kiểm tra IMEI.
- setMaIMEI() cập nhật IMEI sau khi kiểm tra.
- ganSIM() gán số điện thoại.
- goSIM() gỡ số điện thoại.
- setBlacklist() khóa hoặc mở khóa mô phỏng.
- setTrangThai() kiểm tra trạng thái.
- capNhatBTS() cập nhật trạm BTS.
- toFileString() và fromFileString() serialization.
- displayHeader(), displayRow(), displayDetail() hiển thị dữ liệu.

3.4. Kiểm tra IMEI bằng Luhn Mod-10

Trong cấu trúc quốc tế, IMEI gồm 15 chữ số: TAC 8 chữ số, SNR 6 chữ số và check digit 1 chữ số. Chương trình lưu cả chuỗi 15 chữ số làm khóa định danh.

ThietBiIMEI::validateLuhn() thực hiện:

1. Kiểm tra độ dài bằng 15.
2. Kiểm tra tất cả ký tự là chữ số.
3. Từ chối chuỗi toàn số 0.
4. Duyệt chữ số từ phải sang trái.
5. Nhân đôi mỗi chữ số thứ hai tính từ bên phải.
6. Nếu kết quả lớn hơn 9 thì trừ 9.
7. Chấp nhận khi tổng chia hết cho 10.

Thuật toán có độ phức tạp O(n), với n = 15 trong trường hợp IMEI. Constructor và setMaIMEI() cùng gọi validator, nên dữ liệu không hợp lệ không thể đi qua chỉ bằng cách bỏ qua menu.

3.5. Lưu trữ dữ liệu bằng file

3.5.1. Lựa chọn phương án lưu trữ

Bài tập lựa chọn file văn bản thay vì hệ quản trị cơ sở dữ liệu vì:

- không cần cài đặt máy chủ database;
- dễ quan sát bản ghi trong quá trình học tập;
- phù hợp với số lượng dữ liệu nhỏ;
- có thể triển khai bằng thư viện chuẩn;
- dễ kiểm thử trên máy cá nhân.

Hạn chế là toàn bộ dữ liệu được nạp vào std::vector, thao tác ghi thường ghi lại cả collection, khả năng truy cập đồng thời thấp và cần tự kiểm tra tính nhất quán giữa các file.

3.5.2. Định dạng bản ghi IMEI

Mỗi thiết bị được lưu thành bảy trường phân cách bằng ký tự |:

IMEI|Tên thiết bị|Hãng sản xuất|Số điện thoại|Ngày kích hoạt|Trạng thái|Trạm BTS gần nhất

FileIO::split() tách các trường. fromFileString() kiểm tra số lượng trường, chuyển ngày bằng Date::parse() và trả về false nếu dòng không đọc được.

3.5.3. Quy trình đọc và ghi

Khởi động
↓
Đảm bảo thư mục data tồn tại
↓
Đọc từng dòng và trim
↓
Bỏ qua dòng trống hoặc dòng bắt đầu bằng #
↓
Gọi fromFileString()
↓
Đưa bản ghi hợp lệ vào vector
↓
Khi thêm/sửa/xóa: ghi ra file tạm .tmp
↓
Đổi tên file tạm thành file chính

DataStore<T> cung cấp các thao tác add, update, remove, findById, filter, sort, loadFromFile và saveToFile. Khi ghi, hệ thống dùng file tạm trước khi thay file chính để giảm nguy cơ file chính bị ghi dở.

3.6. Luồng xử lý use case IMEI

3.6.1. Thêm thiết bị

Người dùng chọn thêm thiết bị
↓
Nhập IMEI
↓
Kiểm tra độ dài, chữ số và Luhn
↓
Kiểm tra IMEI đã tồn tại chưa
↓
Nhập tên, hãng, số điện thoại, ngày và BTS
↓
Constructor kiểm tra lại các bất biến miền
↓
DataStore::add()
↓
DataStore::saveToFile()
↓
Hiển thị kết quả

3.6.2. Gán hoặc gỡ SIM

Nhập số điện thoại mới
↓
Normalized::isValidPhoneNumber()
↓
ThietBiIMEI::ganSIM()
↓
DataStore::update()
↓
Ghi dữ liệu xuống file

Khi gỡ SIM, goSIM() lưu giá trị ChuaGan.

3.6.3. Khóa thiết bị

Chọn thiết bị theo IMEI
↓
ThietBiIMEI::setBlacklist(true)
↓
Trạng thái chuyển thành KhoaMang
↓
Cập nhật DataStore
↓
Ghi dữ liệu xuống file

Trong mạng thật, EIR có thể có white list, grey list và black list. Trong bài tập, KhoaMang chỉ là trạng thái nghiệp vụ mô phỏng; chương trình không gửi lệnh tới EIR thật.

3.7. Phân tích rủi ro và phương hướng giải quyết

Rủi ro	Nguyên nhân	Phương án xử lý
File dữ liệu chưa tồn tại ở lần chạy đầu	Chưa có thư mục hoặc file data/	DataStore gọi ensureDirectoryExists; tiếp tục với collection rỗng nếu file chưa có
Dòng file sai định dạng	Thiếu trường, sai delimiter hoặc ngày sai	fromFileString() kiểm tra số trường và bắt lỗi parse; bỏ qua bản ghi lỗi
IMEI không hợp lệ	Sai độ dài, có ký tự chữ, toàn 0 hoặc sai checksum	validateLuhn() trả về false; constructor/setter ném InvalidLuhnException
Số điện thoại không hợp lệ	Sai độ dài hoặc đầu số	Normalized::isValidPhoneNumber() và InvalidPhoneNumberException
Mã IMEI hoặc hợp đồng bị trùng	Người dùng thêm lại mã đã tồn tại	DataStore::add() kiểm tra findById() và ném DuplicateIdException
Cập nhật hoặc xóa mã không tồn tại	Nhập sai mã hoặc bản ghi đã bị xóa	NotFoundException và thông báo tại menu
Lỗi khi ghi file	Không có quyền ghi, đường dẫn lỗi hoặc lỗi đổi tên	FileIOException; không báo thành công nếu thao tác ghi thất bại
Chương trình dừng khi đang ghi	Ngắt tiến trình trong lúc ghi	Ghi file .tmp trước rồi thay file chính; đây là biện pháp giảm rủi ro, không phải database transaction đầy đủ
Xóa đối tượng đang được tham chiếu	Hợp đồng hoặc module khác còn dùng số/IMEI	Nhánh hiện tại chưa có ràng buộc liên module đầy đủ; khi tích hợp cần kiểm tra tham chiếu trước khi xóa hoặc chuyển sang trạng thái ngừng dùng
Lộ dữ liệu cá nhân	File chứa số điện thoại	Bản hiện tại chưa có

Rủi ro	Nguyên nhân	Phương án xử lý
	hoặc thông tin khách hàng	xác thực và mã hóa; production cần phân quyền, audit và bảo vệ dữ liệu

3.8. Kiểm thử và kết quả đánh giá

Bộ test trong tests/test_runner.cpp bao gồm:

- Date và năm nhuận.
- Ngày sinh và giới hạn tuổi.
- Số điện thoại di động Việt Nam.
- Các hàm chuẩn hóa chuỗi.
- IMEI và Luhn checksum.
- Vòng đời hợp đồng.
- CRUD và persistence của DataStore<T>.
- Định dạng bảng/card của DisplayHelper.
- Menu hợp đồng và menu thiết bị IMEI.

Lệnh kiểm thử:

make test

Kết quả hiện tại:

Ket qua kiem thu: 139 passed, 0 failed.
TAT CA KIEM THU DA VUOT QUA THANH CONG (100% PASS)!

Kết quả được xác nhận là **139/139 assertion passed**.

Lệnh build:

make all

Kết quả tại thời điểm lập báo cáo:

make: Nothing to be done for 'all'.

Điều này có nghĩa target hiện tại đã up-to-date; đây không phải tuyên bố về một lần clean rebuild mới.

Một số test case tiêu biểu

STT	Tình huống	Kết quả mong đợi
1	IMEI đủ 15 chữ số và đúng Luhn	Thiết bị được khởi tạo thành công
2	IMEI có chữ cái	Bị từ chối

STT	Tình huống	Kết quả mong đợi
3	IMEI toàn số 0	Bị từ chối
4	IMEI sai checksum	Ném InvalidLuhnException
5	Gán số điện thoại 10 chữ số, đúng đầu số	Gán SIM thành công
6	Gán số sai độ dài hoặc sai đầu số	Ném InvalidPhoneNumberException
7	Khóa thiết bị	Trạng thái thành KhoaMang
8	Gỡ SIM	Số điện thoại thành ChuaGan
9	Thêm trùng IMEI	Ném DuplicateIdException
10	Lưu và nạp lại thiết bị	Các trường dữ liệu được khôi phục đúng

Các kiểm thử này chứng minh hành vi source-level và persistence cục bộ. Chúng không chứng minh kết nối với mạng di động thật, khả năng tương thích với EIR production, độ chính xác định vị vô tuyến, khả năng nhiều người dùng đồng thời hoặc mức độ tuân thủ pháp lý khi triển khai dữ liệu thật.

3.9. Cơ hội phát triển hệ thống

- Bổ sung đầy đủ các module gói cước, thuê bao, nạp tiền và hóa đơn.
- Bổ sung nhiều nhà mạng và danh mục đầu số cấu hình được.
- Tách TAC khỏi IMEI để tra cứu hãng, model và cơ quan phân bổ.
- Thêm white list, grey list và lịch sử thay đổi trạng thái EIR.
- Thêm chức năng xuất báo cáo số thiết bị hoạt động, bị khóa, theo hãng hoặc theo BTS.
- Thay file văn bản bằng SQLite hoặc hệ quản trị cơ sở dữ liệu khi dữ liệu lớn.
- Thêm authentication, authorization, audit log và mã hóa dữ liệu nhạy cảm.
- Xây dựng API hoặc giao diện đồ họa trên cùng domain model.
- Bổ sung fixture test, integration test và kiểm thử tải.

3.10. Kết luận chương

Chương 3 đã trình bày giải pháp hướng đối tượng, phân tầng mã nguồn, thiết kế ThietBiIMEI, thuật toán Luhn, định dạng file, luồng xử lý, rủi ro, kiểm thử và hướng mở rộng. Lưu trữ file tạm thời phù hợp với quy mô bài tập, còn các điểm giới hạn được ghi rõ để tránh nhầm lẫn giữa mô phỏng học thuật và hệ thống nhà mạng production.

Chương 4. Kiến thức mới tự tìm hiểu và vận dụng

4.1. Kiến thức mới 1: Xử lý ngoại lệ trong C++

4.1.1. Học gì

Cơ chế exception của C++ sử dụng try, throw và catch để tách phần phát hiện lỗi khỏi phần xử lý lỗi. Các lớp lỗi của dự án kế thừa từ std::runtime_error, sau đó chuyên biệt hóa theo miền nghiệp vụ:

- InvalidLuhnException;
- InvalidPhoneNumberException;
- InvalidDateException;
- DuplicateIdException;
- NotFoundException;
- FileIOException.

4.1.2. Nguồn tìm hiểu

- ISO/IEC 14882, tiêu chuẩn ngôn ngữ C++.
- Tài liệu tham khảo C++ về std::runtime_error và exception handling.
- Tài liệu GNU C++ về biên dịch và chẩn đoán lỗi.
- Mã nguồn hiện tại trong src/lib/shared/Exceptions.h và các model sử dụng exception.

4.1.3. Áp dụng vào đề tài

Khi constructor ThietBiIMEI nhận IMEI sai, chương trình ném InvalidLuhnException. Khi gán số điện thoại sai, chương trình ném InvalidPhoneNumberException. Khi thêm mã đã tồn tại, DataStore<T> ném DuplicateIdException. Menu bắt lỗi và hiển thị thông báo thay vì để chương trình tiếp tục với dữ liệu sai hoặc dừng đột ngột.

4.1.4. Lợi ích

- Dữ liệu sai bị chặn tại đúng lớp sở hữu quy tắc.
- Menu không cần biết chi tiết cách tính Luhn.
- Lỗi được phân loại rõ ràng.
- Có thể kiểm thử từng loại lỗi độc lập.
- Việc mở rộng thêm nghiệp vụ không làm mất cơ chế bảo vệ hiện tại.

4.2. Kiến thức mới 2: Cấu trúc IMEI và thuật toán Luhn

4.2.1. Học gì

IMEI là mã nhận dạng thiết bị di động. Cấu trúc 15 chữ số hiện đại thường gồm TAC, SNR và check digit. Check digit được tính bằng quy tắc Luhn Mod-10. Luhn phù hợp với mã số cố định vì có thể phát hiện nhiều lỗi nhập sai hoặc thay đổi chữ số trong quá trình sao chép.

4.2.2. Nguồn tìm hiểu

- 3GPP TS 23.003 về numbering, addressing and identification.
- 3GPP TS 22.016 về International Mobile station Equipment Identity.
- GSMA TS.06 về quy trình phân bổ và phê duyệt IMEI.

4.2.3. Áp dụng vào đề tài

ThietBiIMEI::validateLuhn() kiểm tra 15 chữ số trước khi constructor hoặc setter chấp nhận IMEI. Bài kiểm thử bao gồm IMEI hợp lệ, chuỗi ngắn, chuỗi dài, chuỗi chứa chữ, chuỗi toàn 0 và chuỗi sai checksum.

4.2.4. Lợi ích

- Không lưu mã IMEI sai checksum.
- Tạo một quy tắc miền có thể tái sử dụng ở menu, model và test.
- Làm cho bài toán gần với nghiệp vụ viễn thông thật hơn.
- Giúp hiểu cách một mã nhận dạng phần cứng được kiểm tra trước khi lưu trữ.

4.3. Kiến thức mới 3: Tách validation khỏi giao diện và dùng template

4.3.1. Học gì

Một hàm kiểm tra thuần túy không nên phụ thuộc vào std::cin hoặc cách người dùng nhập dữ liệu. Template cho phép một logic collection được dùng cho nhiều kiểu thực thể nếu các kiểu đó cung cấp cùng hợp đồng phương thức.

4.3.2. Nguồn tìm hiểu

- ISO/IEC 14882 về class, inheritance và templates.
- Tài liệu C++ standard library về std::vector, std::function và algorithms.
- Tài liệu nội bộ của dự án trong docs/Overview/lib.md và docs/Operational/FUNCTION_CALLFLOW_SPEC.md.

4.3.3. Áp dụng vào đề tài

InputHelper chỉ xử lý nhập và retry. Normalized::isValidPhoneNumber() xử lý kiểm tra số điện thoại độc lập với console. DataStore<T> cung cấp CRUD, filter, sort và persistence cho cả HopDong và ThietBiIMEI.

4.3.4. Lợi ích

- Test có thể gọi trực tiếp validation mà không cần giả lập bàn phím.
- Domain model có thể dùng lại cho API hoặc GUI sau này.
- Giảm code lặp giữa hợp đồng và thiết bị.
- Ranh giới giữa presentation, business và persistence rõ ràng hơn.

4.4. Kiến thức mới 4: Lưu file tạm và kiểm thử hồi quy

4.4.1. Học gì

Ghi trực tiếp lên file chính có thể tạo file dở dang nếu tiến trình bị ngắt. Việc ghi ra .tmp rồi thay file chính giảm nguy cơ này. Khi refactor shared library, test hồi quy là cách nhanh nhất để phát hiện hành vi cũ bị thay đổi ngoài ý muốn.

4.4.2. Nguồn tìm hiểu

- Tài liệu C++ về std::ofstream và std::rename.
- Tài liệu GNU Make về target kiểm thử.
- Mã nguồn DataStore.h, fileio.h/.cpp và tests/test_runner.cpp.

4.4.3. Áp dụng vào đề tài

DataStore<T>::saveToFile() ghi dữ liệu vào file tạm trước khi thay file chính. Bộ test kiểm tra CRUD, duplicate ID, not-found, lưu file và nạp lại file. Sau các thay đổi về ranh giới Normalized và InputHelper, lệnh make test vẫn cho kết quả 139 passed, 0 failed.

4.4.4. Lợi ích

- Giảm nguy cơ mất toàn bộ file vì một lần ghi lỗi.
- Có bằng chứng cụ thể sau mỗi lần thay đổi code.
- Hỗ trợ bảo vệ chất lượng khi nhiều module dùng chung cùng một helper.

4.5. Kết luận chương

Chương 4 đã trình bày bốn nhóm kiến thức được tự tìm hiểu và áp dụng: exception handling, cấu trúc IMEI và Luhn, tách validation khỏi giao diện kết hợp template, và persistence qua file tạm kết hợp regression test. Các kiến thức này không chỉ được mô tả lý thuyết mà đã được thể hiện trong các lớp và test của dự án.

KẾT LUẬN

Bài tập lớn đã xây dựng được phần mềm C++ mô phỏng quản lý hợp đồng dịch vụ và thiết bị đầu cuối trong hệ thống thuê bao di động. Phần thực hiện của sinh viên tập trung vào ThietBiIMEI, thuật toán Luhn Mod-10, gán/gỡ SIM, trạng thái khóa mạng, mã BTS, serialization và các thư viện dùng chung.

Giải pháp phân tầng giúp tách menu console khỏi model nghiệp vụ, tách validation khỏi input và dùng lại logic lưu trữ qua DataStore<T>. Cơ chế exception giúp ngăn dữ liệu sai; cơ chế file tạm giúp giảm nguy cơ hỏng file; test assertion giúp phát hiện hồi quy trong quá trình refactor.

Kết quả kiểm thử hiện tại là **139 passed, 0 failed**, tương ứng 100% bộ test hiện có. Đây là bằng chứng cho phạm vi source-level và persistence cục bộ của bài tập. Để trở thành hệ thống

production, dự án vẫn cần database giao dịch, phân quyền, mã hóa, audit log, tích hợp EIR/thuê bao thật và các kiểm thử quy mô lớn hơn.

TÀI LIỆU THAM KHẢO

1. 3GPP, **TS 23.003: Numbering, addressing and identification**, official specification archive: https://www.3gpp.org/ftp/Specs/archive/23_series/23.003/
2. 3GPP, **TS 22.016: International Mobile station Equipment Identities (IMEI)**, official specification archive: https://www.3gpp.org/ftp/Specs/archive/22_series/22.016/
3. GSMA, **TS.06: IMEI Allocation and Approval Process**, official industry reference: https://www.gsma.com/get-involved/working-groups/gsma_resources/ts-06
4. International Telecommunication Union, **ITU-T E.212: Identification plan for public networks and subscriptions**: https://www.itu.int/rec/T-REC-E.212
5. ISO/IEC, **ISO/IEC 14882:2017 C++ language standard**: https://www.iso.org/standard/68564.html
6. GNU Project, **GNU Make manual**: https://www.gnu.org/software/make/manual/make.html
7. Git documentation, **Git user manual**: https://git-scm.com/docs/user-manual
8. Government of Viet Nam, **Decree No. 13/2023/ND-CP on personal data protection**: https://vanban.chinhphu.vn/?pageid=27160&docid=207759
9. Government of Viet Nam, **Decree No. 49/2017/ND-CP on subscriber information management**: https://vanban.chinhphu.vn/?pageid=27160&docid=189445
10. Local project evidence: README.md, Makefile, docs/Overview/ARCHITECTURE_AND_REQUIREMENTS.md, docs/Overview/lib.md, src/lib/models/imei.h, src/lib/models/imei.cpp, src/lib/shared/, src/lib/menus/, and tests/test_runner.cpp.

Conversion note: This Markdown follows the chapter order of the KTLT assignment brief and the baocaoktlt reference drafts. Convert it to .docx or .pdf for submission; apply the required cover-page, font, spacing, page-number, table-of-contents, and figure formatting in Word/LibreOffice after conversion.